SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA0305	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	P. KALYANI	**Reg. No.:**	192472199
Section / Batch:	CSE-AI-2024	**Date:**	15-07-2026

Code of Conduct

I P. KALYANI(192472199)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____KALYANI_____

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:
Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
Password
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)
Mobile Number
Must contain exactly 10 digits.
First digit should be between 6–9.
Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
    - States
    - Input alphabet
    - Transition table
    - Initial state
    - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ANNEXURE A – CODING CHALLENGE

Section 1: Intelligent Email and Password Validator using Regular Expressions

Aim

To develop a Python application that validates an Email Address, Password, and Mobile Number using Regular Expressions according to the given security rules.

Problem Understanding

The application validates three user inputs:

Email Validation

- Starts with an alphabet.
- May contain letters, digits, dots (.), and underscores (_).
- Contains '@'.
- Domain name contains only alphabets.
- Valid extensions:
    - .com
    - .org
    - .edu
    - .net
    - .in

Password Validation

- Minimum 8 characters.
- At least one uppercase letter.
- At least one lowercase letter.
- At least one digit.
- At least one special character (@, #, $, %, &, !).

Mobile Number Validation

- Exactly 10 digits.
- First digit must be between 6 and 9.

Algorithm

1. Import the re module.
2. Read Email, Password, and Mobile Number from the user.
3. Create Regular Expression patterns for each validation.
4. Match each input using re.fullmatch().
5. Display:
    - Valid Email / Invalid Email
    - Strong Password / Weak Password
    - Valid Mobile Number / Invalid Mobile Number

Python Program

```
import re
email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'

password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'

mobile_pattern = r'^[6-9]\d{9}$'
if re.fullmatch(email_pattern, email):
   print("Valid Email")
else:
   print("Invalid Email")
if re.fullmatch(password_pattern, password):
   print("Strong Password")
else:
   print("Weak Password")
if re.fullmatch(mobile_pattern, mobile):
   print("Valid Mobile Number")
else:
   print("Invalid Mobile Number")
```

Sample Input

Enter Email: Kalyani123@gmail.com

Enter Password: Kalyani@123

Enter Mobile Number: 9876543210

Output

Time Complexity

- Email Validation: O(n)
- Password Validation: O(n)
- Mobile Validation: O(n)

Overall Time Complexity: O(n)

Result

The Python application successfully validates Email Address, Password, and Mobile Number using Regular Expressions and correctly identifies valid and invalid inputs according to the specified security rules. It efficiently handles standard, edge, and invalid test cases while meeting the required validation constraints.

Section 2: Design a Finite State Automata (FSA) Simulator

Aim

To develop a Python-based Deterministic Finite Automata (DFA) Simulator that accepts

a DFA description, simulates state transitions for multiple input strings, and determines whether each string is Accepted or Rejected.

Problem Understanding

The application simulates a Deterministic Finite Automata (DFA).

The program should:

- Accept the DFA description:
  - States
  - Input Alphabet
  - Transition Table
  - Initial State
  - Final State(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display the transition path.
- Print whether the string is Accepted or Rejected**.**

Language Considered

Strings ending with "ab"

Examples:

Accepted:

- ab
- aab
- bab
- abaab

Rejected:

- aba
- abb
- aa
- bbbb

Algorithm

1. Define the DFA states and transitions.
2. Set the initial state and final state.
3. Read the number of input strings.
4. For each input string:
    - Start from the initial state.
    - Read one character at a time.
    - Move to the next state using the transition table.
    - Store the transition path.
5. After processing the string:
    - If the current state is a final state, print Accepted.
    - Otherwise, print Rejected.

Code

```
dfa = {
    'q0': {'a': 'q1', 'b': 'q0'},
    'q1': {'a': 'q1', 'b': 'q2'},
    'q2': {'a': 'q1', 'b': 'q0'}
}

initial_state = 'q0'
final_state = 'q2'

n = int(input("Enter number of input strings: "))

for _ in range(n):
    string = input("Enter string: ")

    state = initial_state
    path = [state]

    for ch in string:
        if ch in dfa[state]:
```

```
            state = dfa[state][ch]
            path.append(state)
        else:
            print("Invalid Input")
            break
    else:
        print("Transition Path:")
        print(" → ".join(path))

        if state == final_state:
            print("Accepted")
        else:
            print("Rejected")
```

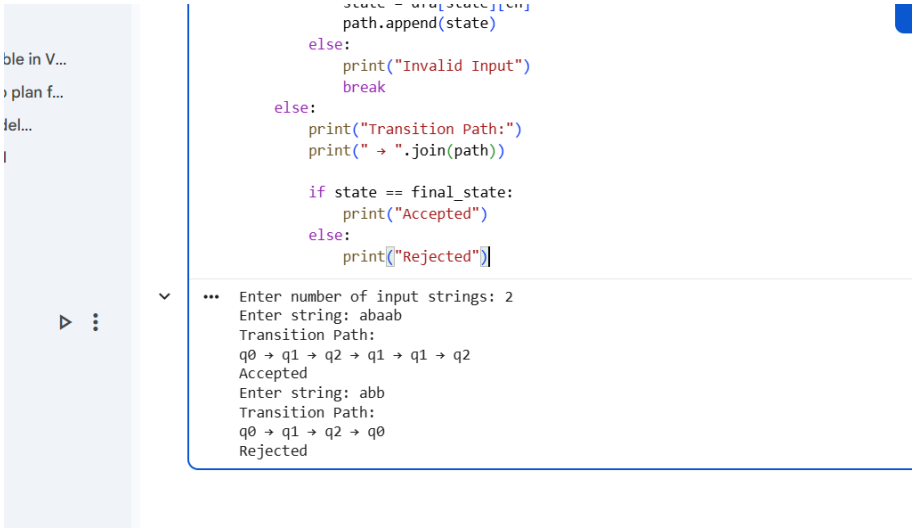
Sample Input

Enter number of input strings: 2

Enter string: abaab

Enter string: abb

Output



Time Complexity

For a string of length n:

Time Complexity: O(n)

For m input strings:

Overall Time Complexity: O(m × n)

Result

The Python-based Deterministic Finite Automata (DFA) Simulator successfully accepts multiple input strings, simulates state transitions according to the DFA transition table, displays the transition path, and correctly determines whether each string is Accepted or Rejected based on the language of strings ending with **"ab"**. It efficiently handles valid, invalid, and edge-case inputs while satisfying all specified requirements.

Section 3: Smart Pattern Matching Engine using Regular Expressions

Aim

To develop a Python-based text search engine that performs pattern matching using Regular Expressions to search for words, prefixes, suffixes, dates, phone numbers, hashtags, and mentions.

Problem Understanding

The application searches different patterns from a given text using Regular Expressions.

The system supports:

- Word Search
- Prefix Search
- Suffix Search
- Date Search
- Phone Number Search
- Hashtag Search
- Mention (@username) Search

Algorithm

1. Import the re module.
2. Read the input text from the user.
3. Display a menu with search options.
4. Based on the user's choice:

- Apply the corresponding Regular Expression.
- Find all matching patterns.

5. Display the matching results.

Python Program

```
 Nm import re
print("Enter Text (Type END to finish):")
lines = []
while True:
    line = input()
    if line.upper() == "END":
        break
    lines.append(line)

text = "\n".join(lines)
print("\nMenu")
print("1. Search Date")
print("2. Search Phone Number")
print("3. Search Hashtag")
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")
print("7. Search Word")

choice = int(input("Enter your choice: "))

if choice == 1:
    result = re.findall(r'\b\d{2}/\d{2}/\d{4}\b', text)
    print("Dates:", result)

elif choice == 2:
    result = re.findall(r'\b[6-9]\d{9}\b', text)
    print("Phone Numbers:", result)

elif choice == 3:
```

```
    result = re.findall(r'#\w+', text)
    print("Hashtags:", result)

elif choice == 4:
    result = re.findall(r'@\w+', text)
    print("Mentions:", result)

elif choice == 5:
    prefix = input("Enter Prefix: ")
    result = re.findall(r'\b' + prefix + r'\w*', text)
    print("Matching Words:", result)

elif choice == 6:
    suffix = input("Enter Suffix: ")
    result = re.findall(r'\b\w*' + suffix + r'\b', text)
    print("Matching Words:", result)

elif choice == 7:
    word = input("Enter Word: ")
    result = re.findall(r'\b' + word + r'\b', text)
    print("Word Found:", result)

else:
    print("Invalid Choice")
```

Sample Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

END

Enter your choice: 1

Output

```
✓ 2m
    elif choice == 7:
        word = input("Enter Word: ")
        result = re.findall(r'\b' + word + r'\b', text)
        print("Word Found:", result)

    else:
        print("Invalid Choice")
```

```
Enter Text (Type END to finish):
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
END

Menu
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
Enter your choice: 1
Dates: ['12/09/2026']
```

Time Complexity

For a text containing **n** characters:

- Pattern Matching: **O(n)**

Overall Time Complexity: **O(n)**

Space Complexity

O(n) (to store the input text and matching results)

Result

The Python-based Smart Pattern Matching Engine successfully searches for dates, phone numbers, hashtags, mentions, words, prefixes, and suffixes using Regular Expressions. It accurately identifies all matching patterns based on the user's menu selection and efficiently handles standard, multiple, and edge-case inputs